

编译原理与 AI 工具链实验报告

0. 实验信息

- 课程：编译原理与 AI 工具链
- 实验主题：在 Cpu0 LLVM17 后端新增 AI 相关整数指令并完成后端优化实验
- 实验环境：WSL Linux
- 工程目录：/home/linea/work/Cpu0_For_LLVM17
- LLVM 版本：llvm-project-llvmorg-17.0.6

1. 实验目标与任务要求

本实验围绕 Cpu0 后端完成三类能力建设：

1. 指令定义与汇编支持 (MC 层)

- 新增 imac/max/min/popc 4 条整数指令
- 支持 llvm-mc 汇编与反汇编

2. 指令选择 (IR -> DAG -> MI)

- $\text{relu}(x) = x > 0 ? x : 0$ 映射为 max
- $\text{fma}(a, b, c) = a * b + c$ 映射为 imac
- popcount 映射为 popc
- $x/10$ 等常量除法走乘高位与移位序列

3. Machine 层优化 Pass

- 新增 Cpu0MulStrengthReduce, 把 mul x, const (非 2 的幂小常量) 改写为 shl + add/sub
- 通过可开关选项验证优化前后差异

2. 获取能编译的基线

执行如下步骤，得到可正常构建的 LLVM+Cpu0 基线工程。

2.1 进入工程并清理旧构建

```
cd ~/work/Cpu0_For_LLVM17/llvm-project-llvmorg-17.0.6
rm -rf build
```

2.2 重新配置 CMake

```
cmake -G Ninja -S llvm -B build \
-D CMAKE_BUILD_TYPE=Release \
-D LLVM_ENABLE_PROJECTS="clang;lld" \
-D LLVM_TARGETS_TO_BUILD="x86;Mips;Cpu0" \
-D LLVM_INCLUDE_TESTS=OFF \
-D LLVM_BUILD_TESTS=OFF \
-D LLVM_INCLUDE_BENCHMARKS=OFF \
-D LLVM_PARALLEL_LINK_JOBS=1
```

2.3 编译基线

```
cmake --build build --parallel $(nproc)
```

2.4 基线结论

- 配置与编译成功。
- 后续所有代码修改均基于该可编译基线进行。

3. 修改代码重新编译

在 gpt-5.3-codex high 的帮助下完成。

3.1 驱动实现的 AI Prompt

课程：编译原理与AI工具链

交付形式：Git 仓库 + 报告 PDF

1. 作业背景与目标

1.1 背景

神经网络在执行过程中（如 ReLU、矩阵乘等计算）通常依赖于 `max`、融合乘加（FMA）等底层指令，当前主流 AI 芯片大多已对这些指令提供了直接硬件支持，而通用 RISC 指令集（如教学用的 Cpu0）通常缺少这些算子的直接支持，导致编译器只能用多条指令拼接，效率低下。

本作业要求你在 [Jonathan Lee 的 Cpu0 LLVM 教程](https://jonathan2251.github.io/lbd/) 基础上，为 Cpu0 后端新增 4 条整数运算指令，并围绕它们完成一组编译优化实验。完成后，你将深入理解 LLVM 后端从 LLVM IR → SelectionDAG → MachineInstr → MC → 汇编 的完整 workflow。

1.2 目标

让 Cpu0 后端能够将以下 C 模式编译为高质量汇编，并通过 lit (LLVM Integrated Tester) 测试。

- `relu(x) = x > 0 ? x : 0` → 单条 `max`
- `fma(a, b, c) = a * b + c` → 单条 `fma`
- `popcount(a)` → 单条 `popc` 指令
- `x / 10` 等常量除法 → `mulh` + 移位

3. 新增指令规格

3.1 指令一览

仅供参考，具体的汇编格式请和已有指令保持一致

助记符

格式

操作

语义

`imac rd, rs, rt, ru`

R4-type

整数融合乘加

$rd \leftarrow (rs \times rt + ru) \bmod 2^{32}$

`max rd, rs, rt`

A-type

有符号最大值

$rd \leftarrow (rs \geq_s rt) ? rs : rt$

`min rd, rs, rt`

A-type
有符号最小值
 $rd \leftarrow (rs \leq rt) ? rs : rt$
popc rd, rs
A-type

统计二进制表示中1的个数

3.2 设计原则

- 复用 Cpu0 已有编码空间：新指令 opcode 从保留区分配
- imac 需要 4 个寄存器操作数 → 新增 R4-type 格式
- max / min/ popc 沿用 A-type: 3 个寄存器操作数，无立即数
- 所有指令不改变 flag、不产生异常

4. 任务1: 指令定义与汇编支持

目标: 让 llvm-mc 能汇编/反汇编 4 条新指令

4.1 任务清单

- 在 Cpu0InstrFormats.td 中新增 R4-type 指令格式类
- 在 Cpu0InstrInfo.td 中定义 4 条指令 (包含 operands / pattern)
- 更新 Cpu0MCCodeEmitter, 支持 R4-type 编码
- 更新 Cpu0InstPrinter, 输出对应汇编
- 更新 Cpu0AsmParser, 以识别新指令

5. 任务2: 指令选择

目标: 让编译器自动将 C 源码下降为新指令

5.1 任务清单

- 在 Cpu0ISelLowering.cpp 将 SMAX/SMIN等标记为 Legal
- 编写 TableGen Pattern 匹配 smax/smin
- 添加 Intrinsics 函数, 支持 popcount 函数, 并匹配到 popc 指令
- 为 imac 编写 DAG Combine

6. 任务3: Machine-Level 优化 Pass

目标: 在 MachineInstr 层实现乘法强度削减 (Multiply Strength Reduction), 理解后端 Pass 的编写框架、指令分析与替换流程。

6.1 背景

Cpu0 指令集中 `MUL` (opcode 0x17) 的调度延迟较高, 而 `SHL` (opcode 0x1e) 和 `ADDu`/`SUBu` (opcode 0x11/0x12) 均为 1 个周期。LLVM 前端 (-O2) 仅将 2 的幂次乘法优化为移位 (如 `x\*8` → `shl x, 3`), 但对于非幂次小常数乘法 (`x\*3`, `x\*5`, `x\*7` 等) 仍然生成 `MUL` 指令, 存在明显的优化空间。

本 Pass 在 MachineInstr 层识别 "常数乘法" 模式, 将其替换为移位 + 加减指令组合, 大幅降低延迟。

6.2 任务清单

- 新建 Cpu0MulStrengthReduce.cpp, 实现 MachineFunctionPass
- 在 Cpu0.h 中声明 createCpu0MulStrengthReducePass()
- 在 CMakeLists.txt 中添加源文件
- 在 Cpu0TargetMachine.cpp 的 addPreEmitPass() 中注册
- 添加 cl::opt 开关 -enable-cpu0-mul-strength-reduce (默认开启)

报告要求

- 添加cpu0后端的过程

现在这台ws1上已经成功运行了

```
cd ~/work/Cpu0_For_LLVM17/llvm-project-llvmorg-17.0.6
```

```
rm -rf build
```

```
cmake -G Ninja -S llvm -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DLLVM_ENABLE_PROJECTS="clang;lld" \
  -DLLVM_TARGETS_TO_BUILD="X86;Mips;Cpu0" \
  -DLLVM_INCLUDE_TESTS=OFF \
  -DLLVM_BUILD_TESTS=OFF \
  -DLLVM_INCLUDE_BENCHMARKS=OFF \
  -DLLVM_PARALLEL_LINK_JOBS=1
```

```
cmake --build build --parallel $(nproc)
```

帮我完成实验任务，并输出一个.md实验文档

3.2 修改后重新编译命令

```
cd ~/work/Cpu0_For_LLVM17/llvm-project-llvmorg-17.0.6
cmake --build build --parallel $(nproc)
```

3.3 与测试配置相关的说明

本地基线使用了 `-DLLVM_INCLUDE_TESTS=OFF`，因此不能直接通过 `llvm-lit` 装载完整测试配置；验证时采用“逐条执行测试文件中的 `RUN` 命令”等价检查方式。

4. 代码实现过程

4.1 任务1：指令定义与汇编支持

4.1.1 新增指令格式与指令定义

1. 在 `Cpu0InstrFormats.td` 中新增 `R4-type`：

- `FrmR4` : `Format<5>`
- `FR4` 编码布局: `opcode|ra|rb|rc|rd|resv8`

2. 在 `Cpu0InstrInfo.td` 中新增指令：

- `IMAC` (opcode `0x53`)
- `MAX` (opcode `0x54`)
- `MIN` (opcode `0x55`)
- `POPC` (opcode `0x56`)

3. 在 `MCTargetDesc/Cpu0BaseInfo.h` 中补充 `FrmR4 = 5`。

4.1.2 新增 `AsmParser` 以支持目标汇编解析

原工程无 `Cpu0 AsmParser`，`llvm-mc` 解析汇编会报“target does not support assembly parsing”。本次新增：

- `llvm/lib/Target/Cpu0/AsmParser/CMakeLists.txt`
- `llvm/lib/Target/Cpu0/AsmParser/Cpu0AsmParser.cpp`
- `llvm/lib/Target/Cpu0/Cpu0Asm.td`
- `llvm/lib/Target/Cpu0/Cpu0RegisterInfoGPRoutForAsm.td`

并在 `llvm/lib/Target/Cpu0/CMakeLists.txt` 中接入：

- `-gen-asm-matcher`
- `add_subdirectory(AsmParser)`
- `LINK_COMPONENTS` 增加 `Cpu0AsmParser`

4.1.3 兼容性修复

`asm-matcher` 生成时报寄存器别名冲突 (`HI/LO` 同名 `ac0`) , 修改为:

- `HI -> "hi"`
 - `LO -> "lo"`
-

4.2 任务2: 指令选择

4.2.1 Legal 设置

在 `Cpu0ISelLowering.cpp` 中把下列操作标记为 `Legal` :

- `ISD::SMAX`
- `ISD::SMIN`
- `ISD::CTPOP`

4.2.2 Pattern 匹配

在 `Cpu0InstrInfo.td` 中新增:

- `smax/smin -> MAX/MIN`
- `ctpop -> POPC`
- `smax x, 0` 和 `smin x, 0` 的匹配 (`relu` 场景)

4.2.3 IMAC DAG Combine

1. 在 `Cpu0ISelLowering.h` 中新增 `Cpu0ISD::IMAC` 。
2. 在 `Cpu0ISelLowering.cpp` 中:

- 为 `ISD::ADD` 打开 DAG Combine
- 识别 `add(mul(a,b), c)` 且 `mul` 单 use
- 生成 `Cpu0ISD::IMAC(a,b,c)` , 再由 Pattern 选为 `imac`

4.2.4 子目标谓词修正

`Cpu0Subtarget.h` 中 `hasChapter12_1()` 原为 `false` , 会导致带 `ch12_1` 谓词指令不可选; 修正为 `true` 。

4.3 任务3: Machine-Level 优化 Pass

4.3.1 新增 Pass 与接入

新增文件:

- `llvm/lib/Target/Cpu0/Cpu0MulStrengthReduce.cpp`

并完成接入:

- `Cpu0.h` 声明 `createCpu0MulStrengthReducePass()`

- `Cpu0/CMakeLists.txt` 增加源文件
- `Cpu0TargetMachine.cpp::addPreEmitPass()` 注册

新增开关:

- `-enable-cpu0-mul-strength-reduce` (默认开启)

4.3.2 变换规则

识别 `mul dst, x, const_reg` (常量来自 `addiu/ori $zero, imm`) 并替换为:

- `2^k`: `shl`
- `2^k + 1`: `shl + addu`
- `2^k - 1`: `shl + subu`
- 负数常量: 末尾补 `subu $zero, result`

另外在可安全场景删除被替换后的死常量装载。

4.3.3 额外稳定性修复

`Cpu0RegisterInfo.cpp::eliminateFrameIndex` 缺少返回值, 补 `return false;`, 避免 `llc` 运行时异常。

5. 关键改动文件清单

- `llvm/lib/Target/Cpu0/AsmParser/CMakeLists.txt` (新增)
- `llvm/lib/Target/Cpu0/AsmParser/Cpu0AsmParser.cpp` (新增)
- `llvm/lib/Target/Cpu0/CMakeLists.txt`
- `llvm/lib/Target/Cpu0/Cpu0.h`
- `llvm/lib/Target/Cpu0/Cpu0Asm.td` (新增)
- `llvm/lib/Target/Cpu0/Cpu0ISelLowering.h`
- `llvm/lib/Target/Cpu0/Cpu0ISelLowering.cpp`
- `llvm/lib/Target/Cpu0/Cpu0InstrFormats.td`
- `llvm/lib/Target/Cpu0/Cpu0InstrInfo.td`
- `llvm/lib/Target/Cpu0/Cpu0MulStrengthReduce.cpp` (新增)
- `llvm/lib/Target/Cpu0/Cpu0RegisterInfo.cpp`
- `llvm/lib/Target/Cpu0/Cpu0RegisterInfo.td`
- `llvm/lib/Target/Cpu0/Cpu0RegisterInfoGPROutForAsm.td` (新增)
- `llvm/lib/Target/Cpu0/Cpu0Subtarget.h`
- `llvm/lib/Target/Cpu0/Cpu0TargetMachine.cpp`
- `llvm/lib/Target/Cpu0/MCTargetDesc/Cpu0BaseInfo.h`
- `llvm/test/MC/Cpu0/ai-insts.s` (新增)
- `llvm/test/CodeGen/Cpu0/ai-patterns.ll` (新增)
- `llvm/test/CodeGen/Cpu0/mul-strength-reduce.ll` (新增)

6. 验证过程与结果

6.1 MC 层验证 (新指令汇编/反汇编)

```
cd ~/work/Cpu0_For_LLVM17/llvm-project-llvmorg-17.0.6
build/bin/llvm-mc -triple=cpu0 -show-encoding llvm/test/MC/Cpu0/ai-insts.s | \
  build/bin/FileCheck llvm/test/MC/Cpu0/ai-insts.s --check-prefix=ENC

build/bin/llvm-mc -triple=cpu0 -filetype=obj llvm/test/MC/Cpu0/ai-insts.s -o - | \
  build/bin/llvm-objdump -d --triple=cpu0 - | \
  build/bin/FileCheck llvm/test/MC/Cpu0/ai-insts.s --check-prefix=DIS
```

结果：imac/max/min/popc 的编码与反汇编均通过检查。

6.2 CodeGen 验证（自动选指令）

```
build/bin/llc -mtriple=cpu0 -verify-machineinstrs -o - \
  llvm/test/CodeGen/Cpu0/ai-patterns.ll | \
  build/bin/FileCheck llvm/test/CodeGen/Cpu0/ai-patterns.ll
```

结果：

- relu -> max
- a*b+c -> imac
- ctpop -> popc
- x/10 走 mult+mfhi+sra 类序列

6.3 Machine Pass 开关验证

```
build/bin/llc -march=cpu0 -mcpu=cpu032II -O2 < llvm/test/CodeGen/Cpu0/mul-
strength-reduce.ll | \
  build/bin/FileCheck llvm/test/CodeGen/Cpu0/mul-strength-reduce.ll --check-
prefix=ON

build/bin/llc -march=cpu0 -mcpu=cpu032II -O2 -enable-cpu0-mul-strength-
reduce=false < llvm/test/CodeGen/Cpu0/mul-strength-reduce.ll | \
  build/bin/FileCheck llvm/test/CodeGen/Cpu0/mul-strength-reduce.ll --check-
prefix=OFF
```

结果：

- 开启：mul3 生成 sh1 + addu
- 关闭：mul3 保持 mul

6.4 综合检查结果

以上命令链路最终得到：ALL_CHECKS_PASSED。

7. 结论

本实验完成了 Cpu0 后端从 LLVM IR -> SelectionDAG -> MachineInstr -> MC -> 汇编 的端到端增强：

- 新增 imac/max/min/popc 并支持汇编/反汇编
- 完成 relu/fma/popcount 目标指令映射
- 完成 Machine 层乘法强度削减 Pass 及可开关验证

- 形成可复现实验流程、测试并上传到仓库 <https://github.com/LineaArpolite/Alcompiler-hw1>
-